

Duale Hochschule Baden-Württemberg Mannheim

Studienarbeit

Übersetzung von Vorlesungsunterlagen in TypeScript – Theoretische Informatik III

Studiengang Informatik

Studienrichtung Angewandte Informatik

Verfasser:	Ertan Ertas und Christian Strerath
Matrikelnummer:	9390224, 4965013
Kurs:	TINF23AI2
Studiengangsleiter:	Prof. Dr. Holger Hofmann
Betreuer:	Prof. Dr. Karl Stroetmann
Bearbeitungszeitraum:	15.10.2025 - 14.04.2026

Erklärung

Wir versichern hiermit, dass wir unsere Studienarbeit mit dem Thema: „Übersetzung von Vorlesungsunterlagen in TypeScript – Theoretische Informatik III“ selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt haben.

Mannheim, 4. März 2026

Ertan Ertas

Kaiserslautern, 4. März 2026

Christian Strerath

Kurzfassung (Abstract)

Auf Deutsch

In English

Inhaltsverzeichnis

1	Einleitung	1
1.1	Problemstellung	1
1.2	Zielsetzung	2
1.3	Methodik und Abgrenzung	3
2	Theoretische Grundlagen	5
2.1	Typentheorie und Datenmodellierung	5
2.1.1	Dynamische Typisierung in Python (Duck Typing)	5
2.1.2	Statische Typisierung und Control Flow Analysis in Type-Script	6
2.1.3	Type Narrowing und Discriminated Unions	7
2.1.4	Strukturelle Typisierung (Structural Typing)	8
2.2	Realisierung von Wertsemantik in Laufzeitumgebungen	10
2.2.1	Vergleich der Implementierungsstrategien	10
2.2.2	Bewertung der Verfahren	11
2.2.3	Anforderungen an Hash-Funktionen	11
2.2.4	Strategien der Kollisionsauflösung	12
2.2.5	Erhalt der Invarianten beim Löschen (Backshifting) . . .	13
3	Literaturverzeichnis	16
A	Anhang	17
A.1	Verzeichnis der portierten Lehrnotebooks	17

Abkürzungsverzeichnis

DHBW Duale Hochschule Baden-Württemberg

WASM WebAssembly

Abbildungsverzeichnis

1	Control Flow Analysis: Der TypeScript-Compiler verfeinert (narrowt) den Typ basierend auf logischen Bedingungen.	6
2	Visualisierung von Separate Chaining	13
3	Visualisierung von Linear Probing	14

Tabellenverzeichnis

1	Asymptotischer Laufzeitvergleich verschiedener Datenstrukturen .	10
2	Übersicht der übersetzten Notebooks im Repository	17

List of Listings

1	Mengen in Python: Das erwartete Verhalten	2
2	Mengen in TypeScript: Das fehlerhafte Verhalten	2
3	Type Narrowing mittels <code>instanceof</code> und Control Flow Analysis . . .	7
4	Implizite Kompatibilität ohne explizite Implementierung	9
5	Algorithmus zur Rückverschiebung (Backshift Deletion), adaptiert nach [5, S. 302]	14

1 Einleitung

Die Theoretische Informatik bildet ein wichtiges Fundament im Informatikstudium. Themen wie Automaten oder formale Sprachen können anfangs jedoch sehr abstrakt wirken. Um diese theoretischen Konzepte greifbarer zu machen, kommen in der Vorlesung „Theoretische Informatik III“ interaktive Notebooks zum Einsatz. Dort wird erklärender Text direkt mit ausführbarem Programmcode gemischt. Bislang basieren diese Notebooks auf der Programmiersprache Python. Studierende können so theoretische Algorithmen direkt ausprobieren, verändern und live nachvollziehen.

Diese Arbeit befasst sich mit der Übertragung dieser Lehrinhalte in die Programmiersprache TypeScript. Der Fokus liegt dabei auf einer Portierung, welche nicht nur die Syntax übersetzt, sondern die Konzepte der formalen Sprachen durch feste, verbindliche Datenstrukturen explizit modelliert.

1.1 Problemstellung

Ausgangspunkt dieser Arbeit sind interaktive Python-Notebooks, die Konzepte der theoretischen Informatik demonstrieren. Werden diese Lehrmaterialien in das TypeScript-Ökosystem übertragen, reicht es jedoch nicht aus, lediglich die Syntax der Sprache zu übersetzen. Es entstehen fundamentale technische Hürden, da beide Sprachen Daten auf unterschiedliche Weise verarbeiten. Folgende zwei Problemfelder stehen dabei im Fokus:

Zu späte Fehlererkennung bei verschachtelten Daten

Python ist eine dynamische Sprache. Das bedeutet, dass Variablen keine festen Typen haben und Fehler oft erst dann auffallen, wenn das Programm ausgeführt wird und abstürzt. Zwar gibt es Werkzeuge, um Typen in Python zu prüfen, in der interaktiven Umgebung eines Jupyter-Notebooks übersehen diese jedoch häufig Fehler.

In der Vorlesung arbeiten wir ständig mit tief verschachtelten Objekten – beispielsweise Datenstrukturen, bei denen eine Liste wieder viele weitere Objekte und Listen enthält. Fehlt einem dieser verschachtelten Elemente versehentlich eine wichtige Eigenschaft, merkt Python das erst, wenn der Code im Hintergrund genau dieses fehlerhafte Element aufruft. Wir benötigen stattdessen eine Sprache, die uns schon während des Tippens garantiert, dass unsere Datenstrukturen über alle Ebenen hinweg fehlerfrei und vollständig aufgebaut sind.

Das unerwartete Verhalten von Mengen (Sets)

Das gravierendste Hindernis bei der Übersetzung zeigt sich jedoch bei der Arbeit mit Mengen (Sets). In der theoretischen Informatik ist es unumgänglich, berechnete Zustände in Mengen zu speichern, um Duplikate zu vermeiden oder später zu prüfen, ob ein Zustand bereits erreicht wurde.

In Python funktioniert das exakt so, wie man es intuitiv erwartet. Fügen wir ein Tupel in eine Menge ein und fragen danach ab, ob ein Tupel mit denselben Zahlen enthalten ist, liefert Python das erwartete Ergebnis:

```
1  menge = set()
2  menge.add((1, 2))
3
4  # Die Abfrage liefert das erwartete Ergebnis
5  print((1, 2) in menge) # Ausgabe: True
```

Listing 1: Mengen in Python: Das erwartete Verhalten

Ein naiver Übersetzungsversuch nach TypeScript führt hier jedoch zu einem fundamentalen Bruch in der Logik. Nutzen wir die Standard-Datenstruktur Set in TypeScript auf die exakt gleiche Weise, erhalten wir plötzlich ein falsches Ergebnis:

```
1  const menge = new Set();
2  menge.add([1, 2]);
3
4  // TypeScript findet das Array überraschenderweise nicht
5  console.log(menge.has([1, 2])); // Ausgabe: false
```

Listing 2: Mengen in TypeScript: Das fehlerhafte Verhalten

Obwohl das Array `[1, 2]` im Code zuvor offensichtlich in die Menge eingefügt wurde, behauptet TypeScript, es sei nicht vorhanden. Dieser zunächst unerklärliche Unterschied führt bei einer direkten Portierung dazu, dass Logik-Abfragen fehlschlagen, Endlosschleifen entstehen und Algorithmen falsche Ergebnisse liefern. Die Ursachenforschung und Behebung dieses Verhaltens ist die zentrale technische Herausforderung dieser Arbeit.

1.2 Zielsetzung

Das übergeordnete Ziel dieser Arbeit ist es, die Lehrinhalte der Vorlesung „Theoretische Informatik III“ vollständig nach TypeScript zu übersetzen. Die neue Co-debasis soll die bisherigen Python-Notebooks ersetzen und dabei die klaren Strukturvorgaben der neuen Sprache nutzen, um Fehler frühzeitig zu vermeiden.

Daraus leiten sich folgende konkrete Teilziele ab:

1. **Strukturierte Neuimplementierung:** Die Algorithmen der Vorlesung sollen in TypeScript portiert werden. Die dynamische, oft fehleranfälligere Natur von Python soll dabei durch verbindliche Datenstrukturen und Typvorgaben ersetzt werden.
2. **Lösung des Mengen-Problems:** Um die Algorithmen fehlerfrei auszuführen, muss das unerwartete Verhalten von TypeScripts Set analysiert und behoben werden. Ziel ist die Entwicklung einer maßgeschneiderten Datenstruktur, die sich exakt wie ein Python-Set verhält und verschachtelte Objekte verlässlich erkennt.
3. **Einheitliche Ausführungsumgebung:** Da die Notebooks lokal ausgeführt werden, muss eine standardisierte Umgebung geschaffen werden, die unabhängig vom Betriebssystem identisch funktioniert. Ein wichtiges technisches Teilziel ist es dabei, die Hintergrundprozesse der Notebooks so anzupassen, dass TypeScripts strenge Prüfregele auch bei der interaktiven, schrittweisen Ausführung zuverlässig erzwungen werden. Der Einrichtungsaufwand für Studierende soll durch eine klare Schritt-für-Schritt-Anleitung minimal gehalten werden.

1.3 Methodik und Abgrenzung

Die methodische Durchführung dieser Arbeit folgt einem fokussierten Ansatz. Der Umfang der Portierung beschränkt sich auf die nummerierten Lehrnotebooks der Vorlesung sowie deren unmittelbare technische Abhängigkeiten. Übungsnotebooks, die primär dem unbetreuten Selbststudium dienen, sind explizit vom Umfang dieser Arbeit ausgeklammert. Die genaue Auswahl der zu übersetzenden Artefakte erfolgte in Abstimmung mit dem Betreuer und ist im Anhang (siehe Abschnitt A.1) tabellarisch aufgeführt.

Ein zentraler methodischer Aspekt betrifft die didaktische Aufbereitung: Die begleitenden Erklärtexpte der Notebooks wurden nicht isoliert übersetzt, sondern inhaltlich an die neuen Strukturen von TypeScript angepasst. Dabei wurde bewusst darauf verzichtet, die neuen Konzepte kontinuierlich mit der alten Python-Implementierung zu vergleichen. Die Notebooks sollen als eigenständiges Lehrmaterial funktionieren, das die theoretischen Konzepte nativ und in sich geschlossen vermittelt.

Auf technischer Ebene war die Minimierung von Systemabhängigkeiten ein wesentliches Entwurfsziel. Insbesondere für die grafische Visualisierung von Automaten und Graphen wurde darauf geachtet, keine komplexen lokalen Installationen

von externer Software vorauszusetzen. Stattdessen kommen integrierbare, plattformunabhängige Lösungen zum Einsatz, um die Ausführung auf verschiedenen Betriebssystemen zu garantieren und den Einstieg für die Studierenden so barrierefrei wie möglich zu gestalten.

Zur Gewährleistung der Reproduzierbarkeit und für die freie Zugänglichkeit der Ergebnisse wird der gesamte entwickelte Quellcode in einem öffentlichen Repository bereitgestellt. Die Implementierung ist unter der URL <https://github.com/cstrerath/formal-languages-typescript> im Verzeichnis TypeScript abrufbar.

2 Theoretische Grundlagen

Dieses Kapitel legt das theoretische Fundament für die Transformationsstrategie. Ein besonderer Schwerpunkt liegt auf der Analyse der Typsysteme, da die Diskrepanz zwischen der dynamischen Natur von Python und der statischen Analyse von TypeScript die zentrale Motivation für die Portierung darstellt.

2.1 Typentheorie und Datenmodellierung

Ein Typsystem ist ein formales logisches System, das Programmkonstrukten Eigenschaften (Typen) zuweist, um das Programmverhalten zu klassifizieren. Nach Pierce gilt ein Programm als typsicher, wenn wohlgetypte Programme nicht in einen undefinierten Zustand geraten („well-typed programs do not go wrong“) [2, S.1ff].

In der Typentheorie wird Typsicherheit häufig durch zwei Eigenschaften charakterisiert [2, S.95ff]:

- **Progress:** Ein wohlgetypter Ausdruck ist entweder bereits ein Wert oder kann einen weiteren Auswertungsschritt durchführen.
- **Preservation:** Wenn ein wohlgetypter Ausdruck einen Auswertungsschritt macht, behält das Resultat diesen Typ bei.

2.1.1 Dynamische Typisierung in Python (Duck Typing)

Python folgt dem Paradigma der dynamischen Typisierung. Typen sind hierbei Eigenschaften von Werten (Laufzeitobjekten), nicht von Variablen. Eine Variable ist lediglich ein Name, der an ein beliebiges Objekt gebunden werden kann. Die Typprüfung erfolgt implizit zur Laufzeit: Eine Operation (z. B. `obj.method()`) ist zulässig, wenn das Objekt zum Zeitpunkt des Aufrufs über das entsprechende Attribut verfügt.

Dieses Konzept wird als **Duck Typing** bezeichnet („If it walks like a duck and quacks like a duck, it is a duck“). Während dies maximale Flexibilität und schnelle Prototypenentwicklung ermöglicht, birgt es im Kontext compiler-naher Softwarearchitekturen signifikante Risiken:

1. **Späte Fehlererkennung:** Inkonsistenzen in Datenstrukturen (z. B. ein fehlendes Feld in einem AST-Knoten) führen erst dann zu einem `AttributeError`, wenn der entsprechende Codepfad zur Laufzeit durchlaufen wird.
2. **Implizite Annahmen:** Die Struktur komplexer Objekte (z. B. „Ein Token ist ein Tupel aus Typ und Wert“) muss durch Dokumentation oder Konvention gesichert werden, da die Sprache sie nicht erzwingt.

Zwar bietet Python mit `mypy` ein Werkzeug für optionale statische Typannotationen, dessen Analysefähigkeit ist jedoch begrenzt. Insbesondere bei der Flussanalyse (Control Flow) kann `mypy` oft nicht automatisch deduzieren, dass ein Typ nach einer Prüfung sicher ist, was zu erzwungenen `cast`-Operationen oder `# type: ignore`-Kommentaren führt.

2.1.2 Statische Typisierung und Control Flow Analysis in TypeScript

Im Gegensatz dazu führt TypeScript eine strikte statische Typprüfung vor der Ausführung durch. Ein zentrales Unterscheidungsmerkmal zu traditionellen statischen Sprachen (wie Java) ist dabei die Fähigkeit zur **Control Flow Analysis (CFA)**. Der Compiler analysiert den logischen Fluss des Codes, um den Typ einer Variable an jeder Stelle im Code so präzise wie möglich zu bestimmen.

Dieses Verfahren wird als **Type Narrowing** bezeichnet. Aus einem breiten Typ (z. B. `string | number`) wird durch Bedingungen ein engerer Typ gefiltert [3].

Abbildung 1 visualisiert diesen Prozess: Der Compiler „versteht“ die `if`-Bedingung und garantiert im jeweiligen Zweig den korrekten Typ, ohne dass zur Laufzeit weitere Casts notwendig sind.

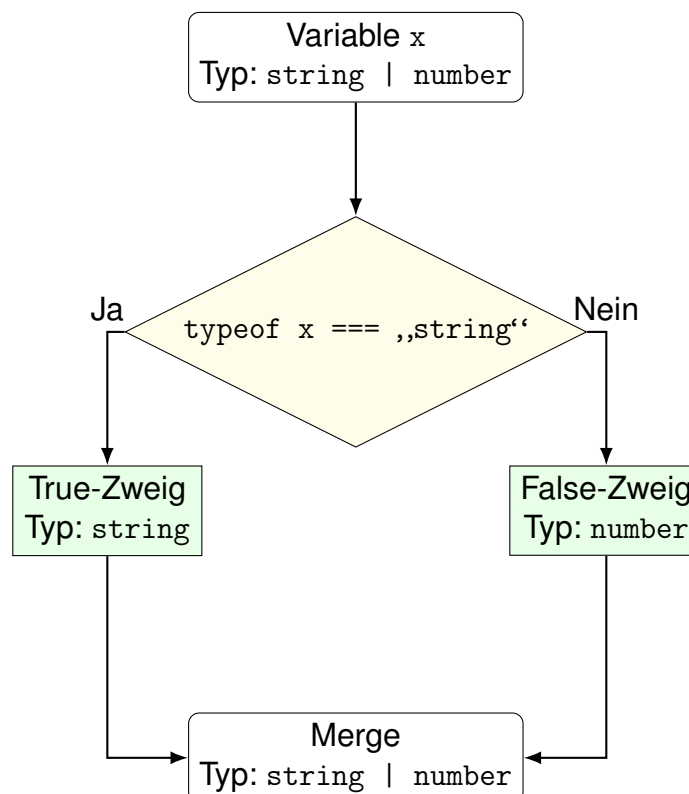


Abbildung 1: Control Flow Analysis: Der TypeScript-Compiler verfeinert (narrowt) den Typ basierend auf logischen Bedingungen.

Quelle: Eigene Darstellung.

2.1.3 Type Narrowing und Discriminated Unions

Eine zentrale Stärke von TypeScript ist die Fähigkeit, Typen innerhalb von Kontrollstrukturen zu verfeinern. Dieses als *Narrowing* bezeichnete Verfahren ermöglicht es dem Compiler, den Typ einer Variable in einem bestimmten Codeblock präzise einzugrenzen.

Besonders relevant ist dies bei der Verwendung von *Union Types* in Kombination mit Klassen. Der Operator `instanceof` fungiert in TypeScript als sogenannter *Type Guard*. Listing 3 illustriert dieses Prinzip anhand einer rekursiven Ausdrucksstruktur.

```
1 class Value {
2     constructor(public val: number) {}
3 }
4
5 class Add {
6     constructor(public left: Expr, public right: Expr) {}
7 }
8
9 type Expr = Value | Add;
10
11 function eval(e: Expr): number {
12     if (e instanceof Value) {
13         return e.val;
14     }
15
16     return eval(e.left) + eval(e.right);
17 }
```

Listing 3: Type Narrowing mittels `instanceof` und Control Flow Analysis

Der Code demonstriert das Zusammenspiel mehrerer TypeScript-Konzepte. Die Klassen `Value` und `Add` definieren die Struktur der Daten, wobei `Value` einen primitiven Wert hält und `Add` rekursiv auf zwei weitere Ausdrücke verweist. Der Typ `Expr` wird als Vereinigung (*Union Type*) dieser beiden Klassen definiert, sodass eine Variable dieses Typs zur Laufzeit entweder eine Instanz von `Value` oder von `Add` sein kann.

Innerhalb der Funktion `eval` dient die Prüfung mittels `instanceof` als Type Guard. Der Compiler erkennt, dass im ersten `if`-Block die Variable `e` zwingend vom Typ `Value` ist, wodurch der Zugriff auf die Eigenschaft `val` statisch erlaubt wird. Im weiteren Verlauf greift die *Control Flow Analysis*: Da der erste Pfad die Funktion durch ein `return` verlässt, schlussfolgert der Compiler für den verbleibenden Code, dass `e` nur noch vom Typ `Add` sein kann. Ein expliziter `else`-Block oder eine

erneute Prüfung sind somit überflüssig; der Zugriff auf `left` und `right` ist typsicher gewährleistet.

Dies unterscheidet sich fundamental von Python, wo ein fehlendes Attribut erst zur Laufzeit einen Fehler verursachen würde. In TypeScript garantiert das Typsystem die strukturelle Integrität der Fallunterscheidung bereits vor der Ausführung.

2.1.4 Strukturelle Typisierung (Structural Typing)

Ergänzend zum Narrowing ist die *strukturelle Typisierung* von TypeScript hervorzuheben. Im Gegensatz zu nominalen Systemen (wie Java oder C#) entscheidet nicht der Name einer Klasse oder eine explizite `implements`-Klausel über die Kompatibilität, sondern ausschließlich die Form (*Shape*) des Objekts.

Sei A der Typ des zuzuweisenden Objekts (Kandidat) und B der erwartete Zieltyp (Interface oder Klasse). Weiterhin sei p eine beliebige Eigenschaft (Property) oder Methode eines Typs. Der Typ A gilt formal als kompatibel zu (oder Subtyp von) B , notiert als $A \subseteq B$, wenn A alle Eigenschaften von B besitzt und deren Typen kovariant sind:

$$A \subseteq B \iff \forall p \in \text{properties}(B) : p \in \text{properties}(A) \wedge \text{type}_A(p) \subseteq \text{type}_B(p) \quad (1)$$

In Worten: Damit A anstelle von B verwendet werden kann, muss A mindestens alle Eigenschaften besitzen, die in B definiert sind. Zusätzliche Eigenschaften in A stören die Kompatibilität nicht.

Listing 4 verdeutlicht dieses Prinzip. Ein Interface definiert lediglich eine Anforderung an die Struktur. Jedes Objekt, das diese Anforderung erfüllt, wird vom Typsystem akzeptiert.

```

1 interface Vector2D {
2     x: number;
3     y: number;
4 }
5
6 function calculateLength(v: Vector2D): number {
7     return Math.sqrt(v.x * v.x + v.y * v.y);
8 }
9
10 class Particle {
11     constructor(
12         public x: number,
13         public y: number,
14         public velocity: number
15     ) {}
16 }
17
18 const p = new Particle(3, 4, 10);
19 const len = calculateLength(p);
20 const len2 = calculateLength({ x: 5, y: 12 });

```

Listing 4: Implizite Kompatibilität ohne explizite Implementierung

Im gezeigten Beispiel wird die Klasse `Particle` (A) als Eingabe von der Funktion `calculateLength` akzeptiert, die `Vector2D` (B) erwartet. Da eine Instanz von `Particle` die geforderten Eigenschaften `x` und `y` besitzt (und mit `velocity` sogar zusätzliche Eigenschaften), gilt die Bedingung $A \subseteq B$ als erfüllt.

Dies erleichtert die Portierung von Python-Code erheblich, da Python's *Duck Typing*-Philosophie („Wenn es wie eine Ente läuft, ist es eine Ente“) direkt auf TypeScript's Interfaces abgebildet werden kann. Ein Algorithmus, der beispielsweise erwartet, dass ein Objekt über eine `hashCode`-Methode verfügt, kann in TypeScript durch ein entsprechendes Interface typisiert werden, ohne dass die verwendeten Klassen in eine starre Vererbungshierarchie gezwungen werden müssen.

Obwohl die explizite Verwendung von `implements` für die Typkompatibilität technisch nicht erforderlich ist, wird sie in der Praxis dennoch empfohlen. Sie verlagert die Fehlererkennung von der Aufrufstelle (*Call Site*) zur Definitionsstelle (*Definition Site*). Würde in der Klasse `Particle` beispielsweise die Eigenschaft `x` versehentlich fehlen, würde der Compiler dies ohne `implements` erst beim Aufruf von `calculateLength(p)` bemängeln. Mit einer expliziten Klausel (`class Particle implements Vector2D`) wird der Fehler direkt an der Klasse markiert, was die Intention des Entwicklers dokumentiert und die Wartbarkeit erhöht.

2.2 Realisierung von Wertsemantik in Laufzeitumgebungen

Ein zentrales Problem bei der Portierung von Python nach TypeScript ist die Diskrepanz im Identitätsbegriff. Während Python für Tupel und unveränderliche Mengen (`frozenset`) automatisch *Wertsemantik* (Value Semantics) anwendet, basieren JavaScript-Container (`Set`, `Map`) bei Objekten strikt auf *Referenzsemantik*. Zwei strukturell identische Zustände eines Automaten gelten in der JavaScript-Laufzeitumgebung als ungleich, sofern sie nicht physikalisch denselben Speicherbereich belegen. Diese Problematik wurde in Abbildung ?? im Abschnitt 1.1 visualisiert.

Um die Algorithmen der theoretischen Informatik korrekt abzubilden, muss daher eine Datenstruktur implementiert werden, die Objekte anhand ihres Inhalts identifiziert. Hierfür existieren in der Informatik verschiedene algorithmische Standardverfahren.

2.2.1 Vergleich der Implementierungsstrategien

Zur Auswahl einer geeigneten Datenstruktur werden im Folgenden drei klassische Ansätze verglichen: Sortierte Arrays, Balancierte Suchbäume sowie Hash-Tabellen.

Ein zentrales Unterscheidungsmerkmal ist die benötigte Schnittstelle der Schlüsselobjekte. Vergleichsbasierte Verfahren setzen voraus, dass Schlüssel eine totale Ordnung besitzen (`Interface Comparable / compareTo`), wohingegen Hash-Verfahren lediglich die Prüfung auf Äquivalenz und die Berechnung eines Streuwertes erfordern (`Interface Equality / hashCode`).

Tabelle 1 fasst das asymptotische Laufzeitverhalten der Operationen *Suchen* und *Einfügen* zusammen. Die Angaben beziehen sich auf N Elemente in der Struktur.

Tabelle 1: Asymptotischer Laufzeitvergleich verschiedener Datenstrukturen

Algorithmus	Worst Case		Average Case	
	<i>Suche</i>	<i>Einfügen</i>	<i>Suche</i>	<i>Einfügen</i>
Geordnetes Array	$O(\log N)$	$O(N)$	$O(\log N)$	$O(N)$
Balancierter Suchbaum	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(\log N)$
Hash-Tabelle	$O(N)$	$O(N)$	$O(1)$	$O(1)$

Quelle: Eigene Darstellung in Anlehnung an [4, S. 520].

2.2.2 Bewertung der Verfahren

Die Analyse der asymptotischen Laufzeiten zeigt deutliche Unterschiede in der Eignung für generische Bibliotheken.

Geordnete Arrays ermöglichen durch die Binäre Suche zwar effiziente Lesezugriffe ($O(\log N)$) und sind extrem speichereffizient, da sie ohne Zeiger auskommen. Sie disqualifizieren sich jedoch für schreibintensive Szenarien. Sobald Mengen dynamisch wachsen, erfordert das Einfügen an einer beliebigen Stelle das Verschieben aller nachfolgenden Elemente im Speicher. Dies führt zu linearen Kosten ($O(N)$). In anwendungsnahen Szenarien, wie etwa einem Backtracking-Algorithmus zum Lösen von Sudokus, werden millionenfach Zwischenzustände in Mengen eingefügt und wieder entfernt. Ein Aufwand von $O(N)$ pro Schritt würde die Gesamtlaufzeit hier exponentiell in die Höhe treiben.

Balancierte Suchbäume garantieren zwar stabile logarithmische Laufzeiten für alle Operationen, weisen jedoch Nachteile auf technischer Ebene auf. Zum einen ist die Speicherstruktur ungünstig für moderne Hardware: Da Knoten einzeln im Heap allokiert und über Zeiger verknüpft sind, liegen logisch benachbarte Daten oft physikalisch weit verstreut. Dies führt beim Traversieren zu häufigen Cache-Misses [5, S. 301]. Zum anderen stellt die Anforderung einer totalen Ordnung (`compareTo`) eine signifikante Hürde dar. Für komplexe, verschachtelte Objekte ist es oft schwierig oder rechenintensiv, eine konsistente Totalordnung zu definieren.

Hash-Tabellen bieten mit einer amortisierten Laufzeit von $O(1)$ die theoretisch beste Performance für große, dynamische Mengen. Das Risiko von Kollisionen existiert, lässt sich durch qualitativ hochwertige Hash-Funktionen aber mitigieren. Entscheidend ist die Einfachheit: Durch den Verzicht auf komplexe Rebalancierungen und die mögliche Nutzung linearer Speicherbereiche profitieren Hash-Tabellen maximal von CPU-Caches [5, S. 301].

2.2.3 Anforderungen an Hash-Funktionen

Eine Hash-Funktion $h : D \rightarrow I$ bildet Elemente aus einer theoretisch unendlichen Domäne D (z. B. alle möglichen Zeichenketten oder Objektgraphen) auf einen endlichen Indexbereich $I = \{0, \dots, m - 1\}$ ab. Damit eine Hash-Tabelle effizient arbeitet, muss die Funktion folgende theoretische Anforderungen erfüllen:

- **Determinismus:** Für zwei identische Objekte x und y muss zwingend gelten: $x = y \implies h(x) = h(y)$. Dies ist trivial für primitive Datentypen, stellt jedoch bei rekursiven Strukturen eine Herausforderung dar. Da Standard-Sets keine definierte Iterationsreihenfolge besitzen, muss die Hash-Funktion kommutativ sein (d. h. unabhängig von der Reihenfolge der Elemente), oder die Elemente müssen vor dem Hashing in eine kanonische Ordnung gebracht werden.

- **Uniformität:** Die Wahrscheinlichkeit, dass ein beliebiger Schlüssel auf einen bestimmten Index abgebildet wird, sollte für alle Indizes gleich sein ($P(h(k) = i) \approx 1/m$). Eine ungleichmäßige Verteilung führt zu Häufungen (Clustering), was die Laufzeit der Hash-Tabelle im schlimmsten Fall auf $O(N)$ degradieren lässt [5, S. 283].
- **Effizienz:** Die Berechnung von $h(x)$ muss schnell erfolgen. In der Praxis kommen oft Verfahren wie *FNV-1a* oder *MurmurHash* zum Einsatz, die durch Bit-Operationen (XOR, Shift) und Multiplikationen mit Primzahlen eine gute Durchmischung bei geringer CPU-Last erreichen.

2.2.4 Strategien der Kollisionsauflösung

Da die Domäne D meist wesentlich größer ist als der Indexbereich I , sind Kollisionen ($h(x) = h(y)$ mit $x \neq y$) nach dem Schubfachprinzip unvermeidbar. Zur Auflösung dieser Konflikte unterscheidet die Literatur zwei Hauptklassen von Verfahren:

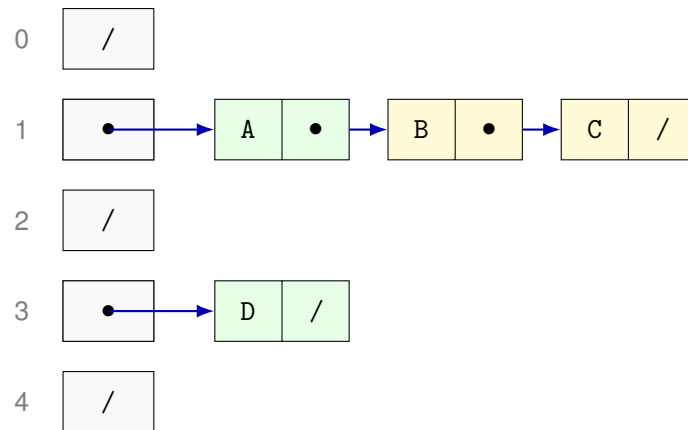
Separate Chaining (Verkettung)

Beim Chaining enthält jeder Slot der Hash-Tabelle einen Zeiger auf eine dynamische Datenstruktur (meist eine einfach verkettete Liste), die alle Elemente aufnimmt, die auf diesen Index hashen (siehe Abbildung 2). Der Vorteil dieses Verfahrens ist seine Robustheit: Der Füllgrad der Tabelle kann theoretisch über 100 % steigen, ohne dass die Tabelle vollständig blockiert. Der Nachteil liegt im Speicheroverhead durch die Zeiger und der schlechten Cache-Lokalität, da die Listen-Knoten verstreut im Heap liegen können.

Open Addressing (Offene Adressierung)

Bei der offenen Adressierung werden alle Elemente direkt im Array der Tabelle gespeichert. Tritt eine Kollision an Index i auf, wird algorithmisch eine alternative Position (Sondierung) bestimmt. Ein weit verbreitetes Sondierungsverfahren ist das *Lineare Sondieren* (Linear Probing), bei dem sequenziell die Positionen $i, i + 1, i + 2, \dots$ geprüft werden (siehe Abbildung 3).

Dieses Verfahren maximiert die Cache-Effizienz, da Datenblöcke sequenziell geladen werden. Es leidet jedoch unter dem Phänomen des *Primären Clusterings*: Belegte Plätze neigen dazu, zu langen Ketten zu verwachsen, was die Wahrscheinlichkeit für weitere Kollisionen in diesem Bereich erhöht und die Performance bei hohem Füllgrad signifikant verschlechtert [5, S. 293].



Legende:

Grün: Listen-Kopf (Direkter Zugriff über Bucket-Pointer).

Gelb: Listen-Rumpf (Zugriff erfordert Traversierung).

Elemente A, B und C teilen sich den Hash-Index 1.

Abbildung 2: Visualisierung von Separate Chaining

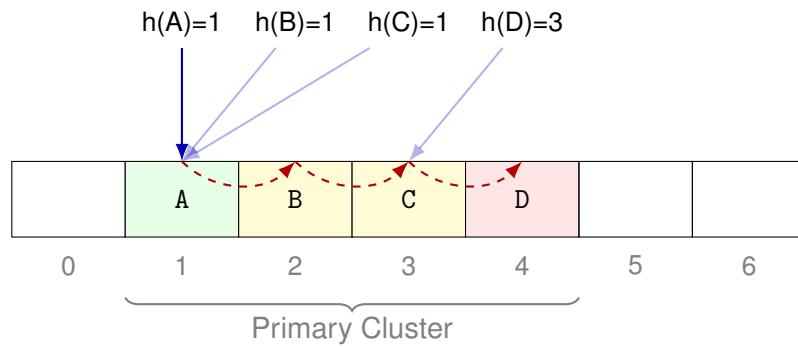
Quelle: Eigene Darstellung.

2.2.5 Erhalt der Invarianten beim Löschen (Backshifting)

Eine besondere Herausforderung bei der offenen Adressierung stellt das Entfernen von Elementen dar. Wird ein Element naiv gelöscht, entsteht eine Lücke (`null`) im Cluster. Nachfolgende Elemente, die aufgrund einer Kollision hinter dieser Lücke platziert wurden, wären bei einer erneuten Suche nicht mehr auffindbar, da der Suchalgorithmus bei der ersten Lücke abbricht.

Klassische Implementierungen verwenden daher oft „Tombstones“ (spezielle Marker für gelöschte Felder). Diese führen jedoch langfristig zu einer Verschmutzung der Tabelle und verlangsamen Fehlsuchen, da der Algorithmus über Tombstones hinwegspringen muss.

Um die Performance auch bei intensiven Modifikationen stabil zu halten, kommt in dieser Arbeit das Verfahren des **Backshift Deletion** (auch bekannt als *Robin Hood Deletion*) zum Einsatz. Listing 5 zeigt eine mögliche Implementierung.



Legende:

A: Optimal platziert (PSL=0) B, C: Verschoben (Kollision) D: Verdrängt durch Cluster

Abbildung 3: Visualisierung von Linear Probing

Quelle: Eigene Darstellung.

```

1 function deleteAtIndex(index: number): void {
2     table[index] = null;
3
4     let hole = index;
5     let scanner = (hole + 1) % capacity;
6
7     while (table[scanner] !== null) {
8         const element = table[scanner];
9         const ideal = hash(element.key);
10
11         const distToHole = (hole - ideal + capacity) % capacity;
12         const distToScanner = (scanner - ideal + capacity) % capacity;
13
14         if (distToHole < distToScanner) {
15             table[hole] = element;
16             table[scanner] = null;
17             hole = scanner;
18         }
19
20         scanner = (scanner + 1) % capacity;
21     }
22 }

```

Listing 5: Algorithmus zur Rückverschiebung (Backshift Deletion), adaptiert nach [5, S. 302]

Der Algorithmus stellt die Invariante des linearen Sondierens wieder her, indem er Lücken schließt. Die Logik lässt sich wie folgt analysieren:

- **Zeile 2:** Das Element an der übergebenen Position wird physisch entfernt. Es entsteht eine Lücke (Variable `hole`).

- **Zeile 5:** Ein zweiter Zeiger (`scanner`) beginnt unmittelbar nach der Lücke und durchläuft zyklisch den restlichen Cluster.
- **Zeile 11–12:** Hier erfolgt die Kernprüfung. Es werden zwei Distanzen berechnet: Wie weit ist die Lücke vom idealen Hash-Index des Elements entfernt (`distToHole`) und wie weit ist die aktuelle Position entfernt (`distToScanner`). Die Modulo-Arithmetik behandelt dabei den zyklischen Überlauf am Array-Ende.
- **Zeile 14:** Ist die Lücke näher an der idealen Position als der aktuelle Platz ($\text{distToHole} < \text{distToScanner}$), bedeutet dies, dass das Element ursprünglich an die Stelle der Lücke (oder davor) wollte, aber verdrängt wurde.
- **Zeile 15–17:** Das Element wird in die Lücke zurückgeschoben. Dadurch wandert die Lücke nun an die Position des Scanners. Der Prozess wiederholt sich rekursiv für die neue Lückenposition, bis das Ende des Clusters erreicht ist.

Die Terminierung des Algorithmus ist durch die Eigenschaften des Linear Probing garantiert, sofern der Füllgrad der Tabelle unter 100 % gehalten wird (typischerweise $\alpha \leq 0.75$). Ein *Cluster* ist in einer Hash-Tabelle definiert als eine zusammenhängende Sequenz von belegten Slots, die von leeren Slots (`null`) begrenzt wird. Da der `scanner`-Zeiger den Cluster ab der Lücke zyklisch durchläuft und sich nur vorwärts bewegt, muss er zwangsläufig das Ende des Clusters erreichen. Da die Tabelle nicht voll sein kann, existiert mindestens ein `null`-Feld, welches die `while`-Schleife in Zeile 7 abbricht. Ein unendlicher Loop ist somit ausgeschlossen.

Das Verfahren hält die Tabelle kompakt und eliminiert die Notwendigkeit von Tombstones vollständig. Zwar sind die Kosten einer einzelnen Löschoperation im Worst-Case höher als beim bloßen Markieren, doch wird dies durch die verbesserte Cache-Lokalität und schnellere Suchzeiten (Misses) bei moderaten Füllgraden kompensiert.

3 Literaturverzeichnis

- [1] Yilin Lu, Shurui Du und Qianwen Wang. „THE AGENTOPIA TIMES: Understanding and Mitigating Hallucinations in Multi-Agent LLM Systems via Data Journalism Gameplay“. In: *2025 IEEE Visualization and Visual Analytics (VIS)*. IEEE, 2025, S. 156–160. DOI: 10.1109/VIS60296.2025.00037.
- [2] Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, 2002. ISBN: 978-0262162098.
- [3] *Narrowing*. TypeScript Team (Microsoft). 2026. URL: <https://www.typescriptlang.org/docs/handbook/2/narrowing.html> (besucht am 08.02.2026).
- [4] Robert Sedgewick und Kevin Wayne. *Algorithmen. Algorithmen und Datenstrukturen*. Person, 2014. ISBN: 978-3-86326-758-2.
- [5] Thomas H. Cormen u. a. *Introduction to Algorithms*. 4. Aufl. MIT Press, 2022. ISBN: 978-0262046305.

A Anhang

A.1 Verzeichnis der portierten Lehrnotebooks

Die folgende Tabelle listet alle Jupyter-Notebooks auf, die im Rahmen dieser Arbeit selektiert und von Python nach TypeScript portiert wurden. Die Gliederung folgt der Kapitelstruktur der Originalvorlesung „Theoretische Informatik III“.

Tabelle 2: Übersicht der übersetzten Notebooks im Repository

Original-Kapitel	Notebook-Dateiname
Chapter 03	01-Ply-Scanning-Example.ipynb 02-Regex-Tutorial.ipynb 03-Exam-Evaluation.ipynb 04-Html2Text.ipynb
Chapter 04 & 05	01-NFA-2-DFA.ipynb 02-Test-NFA-2-DFA.ipynb 03-Regex-2-NFA.ipynb 04-Test-Regex-2-NFA.ipynb 05-DFA-2-RegExp.ipynb 06-Test-DFA-2-RegExp.ipynb 07-Minimize.ipynb 08-Test-Minimize.ipynb 09-Equivalence.ipynb 10-Test-Equivalence.ipynb
Chapter 06	01-Top-Down-Parser.ipynb 02-EBNF-Parser.ipynb
Chapter 07	01-Calculator.ipynb 02-Differentiator.ipynb 03-Interpreter.ipynb
Chapter 10	01-Conflicts.ipynb 02-Conflicts-Resolved.ipynb

Quelle: Eigene Darstellung.